MP 0614 - Despliegue de Aplicaciones Web

UD1: Implementación de Arquitecturas Web

MANUAL UNIFICADO DE ESTUDIO Y REFERENCIA TÉCNICA | DECROLY SANTANDER

SECCIÓN 1.1: INTRODUCCIÓN A LAS ARQUITECTURAS WEB

1. La World Wide Web y Arquitectura Web

La arquitectura de la **World Wide Web (WWW)** proporciona un modelo de programación potente y flexible en el que las aplicaciones y el contenido se presentan en formatos de datos estándar y se localizan mediante navegadores web. Esta infraestructura permite acceder a servicios globales y desarrollar entornos de propósito general.

Escalabilidad y Rendimiento
Capacidad de responder al crecimiento del número de usuarios y carga de tráfico de forma transparente.

Separación de Capas
Aislamiento entre presentación, lógica de negocio y gestión de datos para facilitar el mantenimiento.

Patrones y Portabilidad
Uso de patrones de diseño estandarizados para asegurar la independencia de plataforma y sistema.

2. Esquema de Funcionamiento y Roles

El esquema funcional de los servicios web se fundamenta en la interacción articulada entre tres actores principales:

- **Proveedor de Servicio:** Entidad que diseña, desarrolla e implementa el servicio web, poniéndolo a disposición de forma interna o pública.
- **Consumidor de Servicio:** Entidad o aplicación cliente que accede al servicio web para consumir sus funcionalidades.
- **Agente del Servicio:** Intermediario que publica y facilita la localización y búsqueda dinámica de los servicios.

3. Tecnologías de Ejecución Lado Cliente / Servidor

Tecnologías Lado Servidor
PHP: Lenguaje ejecutable en servidor, amplio uso con MySQL.
ASP / ASP.NET: Entorno de Microsoft para páginas dinámicas sobre IIS.
CGI: Estándar clásico escrito en Perl o C/C++ para procesar peticiones.

Tecnologías Lado Cliente
JavaScript: Lenguaje interpretado por navegadores para interactividad web.
CSS: Hojas de estilo en cascada para separar contenido de diseño.
Java Applets: Ejecución cliente vía JVM (en uso histórico).

SECCIÓN 1.1: ALOJAMIENTO WEB Y CLOUD COMPUTING

1. Modelos de Alojamiento Web (Hosting)

El alojamiento web o *hosting* proporciona la infraestructura necesaria para almacenar y publicar los archivos que componen un sitio web, además de ofrecer gestión de backups, correo y servicios de red.

Hosting Gratuito vs Compartido

Gratuito: Muy limitado en recursos, memoria y soporte. Ideal únicamente para prácticas.
Compartido: Económico (≈4.50€/mes). Servidor compartido entre múltiples clientes isolados. Soporta alto tráfico pero vulnerable a picos de consumo de terceros.

VPS vs Hosting Dedicado

VPS (Servidor Privado Virtual): Acceso exclusivo a una VM. Escalable y flexible pero requiere administración técnica.
Dedicado: Servidor físico exclusivo para grandes corporaciones o proyectos con tráfico masivo.

2. Cloud Hosting y Modelos de Servicio

El Cloud Hosting utiliza máquinas virtuales interconectadas que actúan como un clúster único. Si un nodo físico falla, otro asume el servicio de inmediato sin interrupción (Alta Disponibilidad).

SaaS (Software as a Service)

Aplicaciones listas para usar vía web (ej. Gmail, Dropbox). El usuario no gestiona infraestructura ni código.

PaaS (Platform as a Service)

Proporciona el entorno de ejecución, BBDD y servidores de app (ej. Heroku). El desarrollador solo aporta su código.

IaaS (Infrastructure as a Service)

Capacidad bruta de procesamiento, red y almacenamiento (ej. AWS EC2). El usuario configura el SO y la pila técnica.

3. Tipos de Nubes

Modelo Cloud	Descripción y Caso de Uso	Ejemplos / Proveedores
Cloud Pública	Infraestructura abierta accesible para cualquier usuario o empresa.	AWS, Microsoft Azure, Google Cloud
Cloud Privada	Entorno exclusivo para una sola organización. Máxima seguridad y control.	Infraestructura empresarial propia
Cloud Híbrida	Combina recursos públicos para escalar y privados para datos sensibles.	Arquitecturas bancarias/sanitarias

SECCIÓN 1.2: ARQUITECTURAS ESCALABLES Y TOLERANTES A FALLOS

1. Las 3 Capas Lógicas de una Arquitectura Web

1. Capa de Presentación
Gestiona la interfaz de usuario, navegabilidad, validación de datos de entrada y formateo de salida en el cliente web.

2. Capa de Negocio
Procesa las peticiones, ejecuta las reglas de negocio, coordina la lógica y prepara los resultados para la presentación.

3. Capa de Datos
Sistemas Gestores de Bases de Datos (SGBD) encargados de almacenar, estructurar y recuperar información de forma persistente.

2. Modelos Arquitectónicos

Modelo Cliente-Servidor (2 Niveles)
El cliente solicita recursos directamente a un servidor versátil que asume tanto el procesamiento como el envío de la respuesta.

Arquitectura de 3 Niveles (N-Tier)
Separación física y lógica entre el Cliente, el Servidor de Aplicaciones (Middleware) y el Servidor de Bases de Datos.

3. Estrategias de Escalabilidad y Clústeres

Para gestionar fluctuaciones de tráfico se aplican técnicas de escalado:

- **Escalabilidad Vertical:** Separación física de capas en distintas máquinas especializadas o incremento de hardware (CPU/RAM) de un nodo.
- **Escalabilidad Horizontal:** Clonado de servidores idénticos distribuyendo la carga mediante un balanceador (Software o Hardware L4/L7).
- **Clúster de Servidores:** Grupo de servidores trabajando de forma transparente. Utiliza mecanismos de *replicación de sesión* para garantizar que cualquier nodo pueda atender una petición sin perder el contexto del usuario.

Tipos de Clúster
Clúster de Alta Disponibilidad (HA): Diseñado para garantizar el servicio ininterrumpido eliminando puntos únicos de fallo.
Clúster de Balanceo de Carga: Diseñado para distribuir equitativamente el tráfico entre múltiples instancias.

SECCIÓN 1.3: SERVIDORES DE APLICACIONES

1. Concepto y Necesidad de un Servidor de Aplicaciones

Un **Servidor de Aplicaciones** es un entorno de software que ejecuta la lógica de negocio dinámicamente y proporciona APIs de infraestructura. Se comunica con el cliente a través de redes TCP/IP mediante protocolos interactivos.

¿Por qué usar Servidor de Apps?

- **Alta Disponibilidad:** Operación 24/7 con redundancia y tolerancia a fallos.
- **Escalabilidad:** Crecimiento modular ante demanda creciente.
- **Gestión Centralizada:** Actualizaciones y parches centralizados en el servidor.

¿Cuándo conviene la Instalación Local?

Aplicaciones que requieren consumo intensivo de recursos locales (GPU/CPU) y acceso a periféricos:

- Diseño 3D / CAD (AutoCAD)
- Edición de vídeo (Adobe Premiere)
- Entornos de desarrollo (IDEs)

2. Componentes de la Arquitectura de Aplicaciones Web

1. Cliente Web (Cliente Ligero)

Navegador web que interactúa mediante HTTP/HTTPS procesando HTML/CSS y JavaScript.

2. Aplicación Cliente (Cliente Pesado)

Programa independiente con GUI o consola que se comunica directamente con la app o el servidor.

3. Servidor Web (Contenedor Web)

Gestiona peticiones HTTP y sirve elementos estáticos. Actúa como capa frontal para el usuario.

4. Servidor de Aplicaciones & BBDD

El servidor de app ejecuta la lógica dinámica. La BBDD gestiona el almacenamiento estructurado de datos.

SECCIÓN 1.4: DESCRIPTORES DE DESPLIEGUE Y EMPAQUETADO JAVA

1. ¿Qué es un Descriptor de Despliegue?

Es un archivo de configuración que define exactamente cómo debe implementarse, ejecutarse y configurarse una aplicación web en un contenedor o servidor. En entornos Java EE, el descriptor estándar es el archivo `web.xml`.

```
<?xml version="1.0" encoding="UTF-8"?> <web-app version="3.0"
xmlns="http://java.sun.com/xml/ns/javaee"> <servlet> <servlet-name>CalcuServlet</servlet-name>
<servlet-class>calculadora.CalcuServlet</servlet-class> </servlet> <servlet-mapping> <servlet-
name>CalcuServlet</servlet-name> <url-pattern>/CalcuServlet</url-pattern> </servlet-mapping>
<session-config> <session-timeout>30</session-timeout> </session-config> </web-app>
```

2. Formatos de Archivos para Descriptores

XML
Estándar estructurado y jerárquico tradicional en Java EE (`web.xml`).

YAML
Formatos de serialización limpios y poco verborreicos, estándar en Docker/Kubernetes.

JSON
Formato ligero basado en texto, ampliamente usado en entornos Node.js y APIS.

3. Tipos de Ficheros de Empaquetado en Java

.JAR (Java Archive)
Empaqueta clases compiladas y recursos. Formato ZIP estándar para librerías o ejecutables.

.WAR (Web App Archive)
Empaqueta aplicaciones web completas (JSP, Servlets, HTML, XML y librerías).

.EAR (Enterprise Archive)
Empaqueta múltiples módulos (WAR + JAR) para desplegar en servidores empresariales.

SECCIÓN 1.4: ESTRUCTURA ESTÁNDAR E INSTALACIÓN EN TOMCAT

1. Estructura Interna de un Proyecto Web Java (WAR)

Las aplicaciones web en Java EE deben respetar una estructura de directorios estandarizada (generada automáticamente por IDEs como Eclipse bajo `WebContent`):

```
      Directorio Raíz / (WebContent) ├── Paginas HTML, JSP, CSS, Imágenes └── WEB-INF/ ├── web.xml (Descriptor de despliegue) ├── classes/ (Clases compiladas .class y servlets) └── lib/ (Archivos .jar con librerías adicionales)
```

2. Proceso de Exportación y Despliegue en Tomcat

Para empaquetar en Eclipse: Seleccionar proyecto → Click derecho → `Export > WAR file`.

Paso 1: Configuración de Recursos

El administrador configura DataSources y recursos globales. En Apache Tomcat se realiza mediante los archivos `server.xml` y `context.xml` ubicados en la carpeta `conf/`.

Paso 2: Copia e Instalación

Se copia el archivo `app.war` en la carpeta `webapps/` del servidor Tomcat.

3. Ejecución y Verificación

- **Arranque y Parada:** Se ejecutan los scripts `startup.bat` / `startup.sh` y `shutdown.bat` ubicados en la carpeta `bin/` de Tomcat.
- **Descompresión automática:** Al arrancar, Tomcat descomprime `app.war` creando un directorio `app/` con la estructura interna.
- **Acceso Web:** La aplicación queda accesible desde el navegador mediante la URL: `http://localhost:8080/nombre_app`.

Tendencias de Mercado

Servidores Web: Nginx lidera por alto rendimiento; Apache destaca por flexibilidad; IIS en entornos Windows.
Servidores de Aplicaciones: Tomcat (ligero referente Java), WildFly (anterior JBoss), GlassFish y contenedores (Docker/Kubernetes).